

FICHE DE TD 3/3 • DIFFICILE

Informatique • Tle TI

Programmation d'applications monopostes en langage C

Module 1 : Programmation

Cette fiche propose des problèmes de synthèse mobilisant l'ensemble des notions du module (structures, modularité, algorithmique classique, débogage). Chaque exercice nécessite une conception autonome et une analyse rigoureuse.

EXERCICE 1 Calculatrice modulaire à mémoire

On souhaite concevoir une calculatrice scientifique en ligne de commande gérant une mémoire unique (type `double`) initialisée à 0. Le programme principal affiche un menu (via `switch`) et appelle des fonctions dédiées. Les opérations binaires (addition, soustraction, multiplication, division, puissance) utilisent la mémoire comme opérande gauche et une valeur saisie comme opérande droit ; le résultat devient la nouvelle mémoire. Les opérations unaires (racine carrée, cosinus, valeur absolue, partie entière) s'appliquent sur la mémoire.

1. Définir une structure `Calculatrice` regroupant la mémoire courante et un compteur d'opérations effectuées. Justifier l'intérêt de passer cette structure par adresse aux fonctions de calcul plutôt que par valeur.
2. Écrire le prototype et l'en-tête de la fonction `void operationBinaire(Calculatrice* c, char op, double val)`. Elle doit mettre à jour la mémoire et incrémenter le compteur. Gérer la division par zéro et les erreurs de domaine (ex : racine négative) sans planter le programme, en affichant un message d'erreur et en laissant la mémoire inchangée.
3. Proposer une implémentation de la fonction `double puissance(double base, int exp)` récursive (exposant entier positif ou nul) et l'intégrer au `switch` (choix 'p'). Comparer brièvement l'efficacité en pile d'exécution avec une version itérative.
4. Le programme doit permettre de chaîner les opérations jusqu'à la saisie de 'q' (quitter). Écrire la boucle principale `do...while` dans `main`, incluant l'affichage formaté de la mémoire (`% .4f`) et du compteur avant chaque menu.

EXERCICE 2 Traitement de notes avec tri et recherche

On gère les notes d'une classe de N élèves ($N \leq 50$). Chaque élève est identifié par un matricule (entier strictement positif) et une note (réel dans $[0, 20]$). On utilise un tableau de structures `Etudiant`.

1. Écrire la fonction `void saisieClasse(Etudiant t[], int* n)` qui remplit le tableau et modifie `n` par adresse. Vérifier l'unicité des matricules (recherche séquentielle à

chaque saisie) et la validité des notes.

2. Implémenter le tri par sélection itératif `void triSelection(Etudiant t[], int n)` triant le tableau par note décroissante (et par matricule croissant en cas d'égalité). Détail : écrire explicitement l'échange de deux structures via une variable temporaire de type `Etudiant`.
3. Écrire une fonction `int rechercheSequentielle(const Etudiant t[], int n, int mat)` renvoyant l'indice de l'élève de matricule `mat` ou `-1`. Pourquoi le paramètre `t` est-il qualifié `const` ici ?
4. On souhaite déterminer la médiane des notes. Proposer une fonction `double mediane(const Etudiant t[], int n)` utilisant le tableau déjà trié. Gérer les cas n pair et impair. Quelle est la complexité en temps de cette fonction ?
5. (Ouvert) Si N devient très grand (ex : 10 000), le tri par sélection reste-t-il pertinent ? Citer un algorithme de tri plus efficace vu en cours ou connu, et donner sa complexité moyenne.

EXERCICE 3 Analyse de code, débogage et récursivité terminale

On fournit le code C suivant, destiné à calculer le PGCD de deux entiers positifs par l'algorithme d'Euclide (version récursive) et à afficher les étapes. Le code contient plusieurs erreurs (syntaxe, logique, conception) et ne compile pas sous `gcc -Wall -std=c99`.

```
#include <stdio.h>
#include <math.h>

int pgcd_rec(int a, int b) {
    if (b = 0) return a;
    printf("Etape: pgcd(%d, %d)\n", a, b);
    return pgcd_rec(b, a % b);
}

void main() {
    int x, y;
    printf("Entrez deux entiers: ");
    scanf("%d %d", x, y);
    if (x < 0 || y < 0) {
        printf("Erreur: positifs uniquement");
        return 0;
    }
    int res = pgcd_rec(x, y);
    printf("PGCD = %d\n", res);
}
```

1. Lister précisément toutes les erreurs de compilation (syntaxe, types, bibliothèques) et les erreurs logiques/exécution détectées à la lecture. Pour chacune, donner la correction exacte.
2. La fonction `pgcd_rec` est-elle récursive terminale? Justifier. Si non, réécrire une version `pgcd_term(int a, int b, int acc)` (ou équivalent) qui soit récursive terminale, en expliquant l'intérêt pour la pile d'exécution.
3. Modifier le `main` corrigé pour qu'il boucle tant que l'utilisateur ne saisit pas deux zéros (00), en utilisant une structure `while` appropriée. Ajouter une gestion robuste de `scanf` (test du nombre d'éléments lus) pour éviter une boucle infinie sur entrée invalide (ex : lettre).
4. (Analyse critique) L'inclusion de `math.h` est-elle nécessaire ici? Quel fichier d'en-tête standard aurait dû être inclus pour `printf/scanf`? Expliquer le rôle de l'édition de liens (linker) vis-à-vis de `math.h` si on utilisait `sqrt`.